Floating point error

This page maybe follows from Points on floats

I ran into trouble trying to understand floating point error. After reading Wikipedia floating point, Wikipedia machine epsilon and What every computer scientist should know about floating point, I felt the need of some more explanation, and so here it is.

Units at the last place

Taking the notation from Every computer scientist; let's imagine we have a floating point number that has base 10 and 3 digits in the significand, say 3.14×10^1. Because we only have 3 digits, the nearest larger number that we can represent is obviously 3.15×10^1. This number differs from 3.14×10^1 by one unit in the last place (ULP). Any real number z that is between 3.14×10^1 and 3.15×10^1 can at best be represented with one of these two numbers. Let's say z is actually π; now $3.1415926\ldots$ is best represented in our numbers as 3.14×10^1, and the rounding error is $\pi - 3.14 \times 10^1 = 0.0015926\ldots$ In the worst case, we could have some real number 3.145×10^1 that will have rounding error 0.005. If we always choose the floating point number nearest to our real number z then the maximum rounding error occurs when z is halfway between two representable numbers; in that case the rounding error is 0.5 ULP.

We can generalize to floating point numbers of form:

$$d_1 . d_2 \ldots d_p \times \beta^e$$

Where p is the number of digits in the significand, β is the *base* (10 in our example), and e is the exponent.

1 ULP corresponds to:

$$0.00\ldots1 \times \beta^e$$

where there are $p - 1$ zeros in the significand. This is also:

> ### Note:
>
> Normalized representation in floating point
>
> Short version: The floating point representation of a number is *normalized* if d_1 is not zero.
>
> Long version: consider the number 1.00 represented in the $p = 3, \beta = 10$ system that we started with. We can represent this number as 1.00×10^0 or 0.10×10^1 or 0.01×10^2. The *normalized* representation is the representation with a non-zero first digit - 1.00×10^0 in this case. There is only one normalized representation of a number in a particular floating point representation, so a normalized representation is unique.

$$1.0 \times \beta^{e-(p-1)}$$

Note that any normalized floating point number with exponent e has the same value for 1 ULP. Let's define:

$$ulp(e, p) \rightarrow \beta^{e-(p-1)}$$

We can represent any real number x in normalized floating point format by using an infinite significand:

$$d_1 . d_2 \ldots \times \beta^e$$

Again, *normalized* means that $d_1 \neq 0$. The ULP value for a real value x in some some finite floating point format is still $ulp(e, p)$ where p is the number of digits in the significand as above.

Absolute error

The IEEE standard for floating point specifies that the result of any floating point operation should be correct to within the rounding error of the resulting number. That is, it specifies that the maximum rounding error for an individual operation (add, multiply, subtract, divide) should be 0.5 ULP.

In practice it's now very hard indeed to find a machine that does not implement this rule for floating point operations.

Imagine we have two finite floating point numbers q and r and we combine them using one of the operators {`+, -, *, /`} in a perfect world at infinite precision:

$$x = q \circ r$$

where $\circ$ is one of the operators {`+, -, *, /`}. Let's call the actual finite precision number returned from this calculation $fl(x)$. The IEEE standard specifies that $fl(x)$ should be the closest number to x that can be represented in the finite precision format.

Note:

What is the floating point exponent for any given real number?

So far we've assumed that we know the representation of our floating point number in terms of significand and exponent.

But — what if we have a some infinite precision number x and we want to know how to represent it in floating point?

A simple algorithm might be to get the exponent by an algorithm like this:

```
x1 = abs(x)
e1 = logB(x1)
exponent = floor(e1)
```

Where $abs(y)$ gives the absolute value of y, $logB(y)$ is the log to base β, and $floor(y)$ gives the most positive integer i, such that $i <= y$ [1].

We can then get the mantissa part with $round(x / \beta^{e2}, p - 1)$, where $round(y, z)$ rounds the number y to z digits after the decimal point.

Worked example in Python with our original system of $p = 3, \beta = 10$:

```
from math import abs, log10, floor, round
x = -0.1234 # a number with greater precision than format allows
p = 3 # number of digits in mantissa
x1 = abs(x) # 0.1234
e1 = log10(x1) # -0.9086848403027772
exponent = floor(e1) # -1
m1 = x / (10 ** e2) # -1.234
mantissa = round(m1, p-1) # -1.23
```

giving -1.23×10^{-1} as the floating point representation.

For full accuracy, the algorithm has to be a little more sophisticated than this, but this is a reasonable first pass [2].

We remember that p is the number of digits in the significand in our finite floating point format. The IEEE rule then becomes:

$$|fl(x) - x| \leq 0.5 \times ulp(e, p)$$
$$|fl(x) - x| \leq 0.5 \times \beta^{e-(p-1)}$$

Relative error

The *relative error* is the rounding error divided by the infinite precision real number x:

$$\left|\frac{fl(x) - x}{x}\right| \leq \frac{0.5 \times \beta^{e-(p-1)}}{x}$$

However, any value for x that has some exponent e has the same value for $ulp(e, p) = \beta^{e-(p-1)}$. Let m be the largest digit in base β; thus $m = \beta - 1$. For example $m = 9$ in base 10 ($\beta = 10$). The values of x between $1.0 \times \beta^e$ and $m\,.\,mmm\ldots \times \beta^e$ all have the same value for 1 ULP = $\beta^{e-(p-1)}$. The *relative* rounding error will be greater for smaller x with the same exponent. Let:

$$a = 0.5 \times ulp(e, p)\,.$$
$$a = 0.5 \times \beta^{e-(1-p)}$$

Make x the smallest value with this exponent that has a large rounding error:

$$x = 1.0 \times \beta^e + a$$

The relative rounding error ϵ is:

$$\epsilon = \frac{a}{\beta^e + a}$$

Because a is very small compared to β^e:

$$\epsilon \approx \frac{0.5 \times \beta^{e-(p-1)}}{\beta^e}$$
$$\epsilon \approx 0.5 \times \beta^{1-p}$$

Now make x the largest value with this exponent and that has a large rounding error:

$$x = m\,.\,mm\ldots \times \beta^e - a$$
$$x \approx 1.0 \times \beta^{e+1} - a$$

then:

$$\epsilon \approx \frac{a}{\beta^{e+1} - a}$$
$$\epsilon \approx \frac{0.5 \times \beta^{e-(p-1)}}{\beta^{e+1}}$$
$$\epsilon \approx 0.5 \times \beta^{-p}$$

So, the *maximum* relative error for x varies (depending on the value of x) between $\approx 0.5 \times \beta^{-p}$ and $\approx 0.5 \times \beta^{1-p}$.

Therefore the relative error for any x (regardless of exponent) is bounded by the larger of these two maxima:

$$\epsilon \leq 0.5 \times \beta^{1-p}$$

Machine epsilon

Now note that β^{1-p} is the ULP for 1; that is $1.0 \times \beta^{e-(p-1)}$ where e is 0. Some people refer to this value as *machine epsilon*, others use that term for $0.5 \times \beta^{1-p}$ - see variant definitions. MATLAB and Octave return β^{1-p} from their `eps()` function. numpy uses the same convention in its `np.finfo` function. For example, the standard `float64` double precision type in numpy has $\beta = 2; p = 53$:

```
>>> import numpy as np
>>> np.finfo(np.float64).eps == 2**(1-53)
True
```

Thanks to

Stefan van der Walt for several useful suggestions and corrections.

Footnotes

[1] See Wikipedia floor / ceiling functions. The floor function here (and in C and Python and the Wikipedia page) returns the integer closest to negative infinity. For example, `floor(1.9) == 1`, `floor(-1.1) == -2`.

[2] To find the exact closest floating point representation of a given number, we have to take into account that large values with exponent e may in fact be closer to $1 \times \beta^{e+1}$. For example, with $p = 3$, $\beta = 10$, the infinite precision value 9.996 is closer to 1.00×10^1 than 9.99×10^0, even though `floor(log10(9.996)) == 0`.